Konzept: Deputat-Speicherung pro Untis-Periode statt pro Monat

> **Status:** v0.3 (2026-04-29) — Korrektur-Layer (§11) ergänzt. Phase 1 implementiert + getestet.
> **Beispiel-Lehrer:** Bergmann Benjamin (UntisTeacherID 166, GYM) **Schuljahr:** 2025/2026

---

0. Leitprinzip

> **Untis ist die einzige Quelle der Wahrheit.** Die App spiegelt die Untis-Periodendaten 1:1, bereitet sie für Personalverwaltung & Bezirksregierungs-Statistik auf und macht sie tagesgenau auswertbar — ohne eigene Logik draufzulegen, die Untis-Information verändert.
>
> **Konsequenzen:**
>
> - Untis erlaubt Werteänderungen nur zum Montag → unsere Periodendaten sind ohnehin tagesgenau ableitbar.
> - Wir denken **niemals** Daten dazu, die nicht aus Untis kommen.
> - Wir werfen **niemals** Daten weg, die Untis liefert (kein Coverage-Tie-Breaker mehr).
> - Lücken in Untis-Daten (z. B. Sommerferien zwischen Schuljahren) füllen wir durch **Fortschreiben des letzten bekannten Werts** — nicht mit 0, nicht mit Schätzungen.

---

1. Worum geht's

Aktuell speichert die App pro Lehrer **einen Wert pro Monat** (`deputat_monatlich`). Wenn in einem Monat **mehrere Untis-Perioden** liegen, gewinnt diejenige mit der größten Tages-Coverage — die Werte der anderen Perioden gehen verloren. Tagesgenaue Berechnung ist dadurch nicht mehr möglich.

Lösung: Speichere den Wert in der natürlichen Untis-Granularität — **eine Zeile pro** `(Lehrer, Periode)` . Untis liefert das ohnehin so.

---

2. Bergmann Benjamin — was Untis sagt

Aus der Untis- `Teacher` -Tabelle (`PlannedWeek` / 1000 = Wochenstunden):

Term	Periode	Gilt von	Gilt bis	**Wochenstd.**
1	Periode1	25.08.2025	14.09.2025	8,0
2	Periode2	15.09.2025	21.09.2025	8,0
3	Periode3	22.09.2025	05.10.2025	8,0
4	Periode4	06.10.2025	19.10.2025	8,0
5	Periode5	20.10.2025	02.11.2025	8,0
6	Periode6	03.11.2025	09.11.2025	**10,0 ↑**
7	Periode7	10.11.2025	16.11.2025	10,0
8	Periode8	17.11.2025	23.11.2025	10,0
9	Periode9	24.11.2025	04.01.2026	10,0
10	Periode10	05.01.2026	08.02.2026	10,0
11	Periode11	09.02.2026	01.03.2026	**17,0 ↑**
12	Periode12	02.03.2026	15.03.2026	17,0
13	Periode12b	16.03.2026	12.04.2026	17,0
14	Periode13	13.04.2026	19.04.2026	**10,0 ↓**
15	Periode14	20.04.2026	26.04.2026	10,0
16	Periode14b	27.04.2026	03.05.2026	10,0
17	Periode15	04.05.2026	24.05.2026	**16,0 ↑**
18	Periode16	25.05.2026	19.07.2026	16,0

Eigentliche Wertwechsel (Sicht des Personalsachbearbeiters):

#	Wirksam ab	Term-Wechsel	Alt → Neu	Δ
1	**Mo 03.11.2025**	T5 → T6	8,0 → 10,0	+2,0
2	**Mo 09.02.2026**	T10 → T11	10,0 → 17,0	+7,0
3	**Mo 13.04.2026**	T13 → T14	17,0 → 10,0	-7,0
4	**Mo 04.05.2026**	T16 → T17	10,0 → 16,0	+6,0

Genau **4 Änderungen über das Schuljahr** — alle wirksam zum Montag. Diese 4 Daten will der Sachbearbeiter im UI sehen.

---

3. Was die App aktuell daraus macht (HJ 2026 für Bergmann)

Monat	DB speichert	"Sieger-Periode"	Tagesgenau (wahr)	Differenz
Jan	10,0	T10 (27 Tage)	10,000	0 ✓
Feb	**17,0**	T11 (20 Tage)	**15,000**	**+2,0** ❌
Mär	17,0	T13 (16 Tage)	17,000	0 ✓
Apr	**17,0**	T13 (12 Tage)	**12,800**	**+4,2** ❌
Mai	**16,0**	T17 (21 Tage)	**15,419**	**+0,58** ❌
Jun	16,0	T18 (30 Tage)	16,000	0 ✓
Jul	16,0	T18 fortgeschrieben	16,000	0 ✓

> **Juli/August — Fortschreibe-Regel:** T18 endet am 19.07., danach Sommerferien bis ~24.08. (Beginn T1 SY 2026/27). Wir **schreiben den letzten bekannten Wert (16,0) bis zum nächsten Periodenstart fort**. Juli und August sind beide voll 16,0 WS — fachlich korrekt, weil der Lehrer auch in den Ferien sein Deputat behält.
>
> Σ Fehler ohne neues Modell für Bergmann in HJ 2026: **6,8 Wochenstunden** (Feb +2,0 / Apr +4,2 / Mai +0,58).

---

4. Vorgeschlagenes Datenmodell

4.1 Neue Tabelle `untis_terms` (Master-Periodendaten)

```
CREATE TABLE untis_terms (
    school_year_id  INTEGER NOT NULL,           -- z.B. 20252026
    term_id         INTEGER NOT NULL,           -- 1..18
    term_name       VARCHAR(50),                -- "Periode12b"
    date_from       DATE NOT NULL,              -- echtes DateFrom aus Untis
    date_to         DATE NOT NULL,              -- effektives DateTo (LEAD(DateFrom)-1 oder echtes
    is_b_period     BOOLEAN DEFAULT false,      -- true wenn Untis ein echtes (nicht Schuljahresend
    sync_datum      TIMESTAMPTZ,
    created_at      TIMESTAMPTZ DEFAULT now(),
    updated_at      TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (school_year_id, term_id),
    CHECK (date_from <= date_to)
);
```

Befüllung: separater n8n-Knoten (oder Sub-Query im bestehenden Workflow #223), der die Untis-`Terms`-Tabelle 1:1 spiegelt — pro Schuljahr ca. 15–20 Zeilen.

4.2 Neue Tabelle `deputat_pro_periode` (Lehrer-Werte pro Periode)

```
CREATE TABLE deputat_pro_periode (
    id                    SERIAL PRIMARY KEY,
    lehrer_id             INTEGER NOT NULL REFERENCES lehrer(id),

    -- Periodenidentität (FK auf untis_terms zusammengesetzt)
    untis_schoolyear_id   INTEGER NOT NULL,
    untis_term_id         INTEGER NOT NULL,

    -- Datums-Cache (entkoppelt für schnelle Range-Queries, redundant zu untis_terms)
    gueltig_von           DATE NOT NULL,
    gueltig_bis           DATE NOT NULL,

    -- Werte (genau wie heute in deputat_monatlich)
    deputat_gesamt        NUMERIC(8,3) NOT NULL,
    deputat_ges           NUMERIC(8,3) NOT NULL DEFAULT 0,
    deputat_gym           NUMERIC(8,3) NOT NULL DEFAULT 0,
    deputat_bk            NUMERIC(8,3) NOT NULL DEFAULT 0,

    -- Stammschule pro Periode (zukunftssicher: Lehrer wechselt mitten im Jahr)
    stammschule_code      VARCHAR(10),

    -- Audit
    quelle                VARCHAR(20) DEFAULT 'untis',
    sync_datum            TIMESTAMPTZ,
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at            TIMESTAMPTZ NOT NULL DEFAULT now(),

    UNIQUE (lehrer_id, untis_schoolyear_id, untis_term_id),
    FOREIGN KEY (untis_schoolyear_id, untis_term_id)
        REFERENCES untis_terms(school_year_id, term_id)
);

CREATE INDEX idx_dpp_lehrer ON deputat_pro_periode(lehrer_id);
CREATE INDEX idx_dpp_zeitraum ON deputat_pro_periode USING GIST (
    daterange(gueltig_von, gueltig_bis, '[]')
);
```

Für Bergmann sind das **18 Zeilen** pro Schuljahr.

4.3 Tagesgenaue Monatswerte als View — mit Fortschreibung

Der View hat **zwei Stufen**:

1. `v_deputat_pro_tag` weist jedem Tag den geltenden Wert zu (mit Fortschreiben bei Lücken).
2. `v_deputat_monat_tagesgenau` aggregiert daraus den Monatsmittelwert über die Kalendertage.

```
-- Stufe 1: Pro Tag den geltenden Wert (mit Fortschreibung des letzten Werts in Lücken)
CREATE OR REPLACE VIEW v_deputat_pro_tag AS
WITH tage AS (
    SELECT generate_series(
        (SELECT MIN(gueltig_von) FROM deputat_pro_periode),
        (SELECT MAX(gueltig_bis) FROM deputat_pro_periode) + INTERVAL '90 days',
        INTERVAL '1 day'
    )::date AS tag
),
lehrer_aktiv AS (
    SELECT DISTINCT lehrer_id FROM deputat_pro_periode
)
SELECT
    t.tag,
    la.lehrer_id,
    -- Letzte Periode <= aktueller Tag: deren Wert gilt (auch in Ferienlücken)
    (SELECT dpp.deputat_gesamt
     FROM deputat_pro_periode dpp
     WHERE dpp.lehrer_id = la.lehrer_id
       AND dpp.gueltig_von <= t.tag
     ORDER BY dpp.gueltig_von DESC
     LIMIT 1)                                AS deputat_gesamt,
    (SELECT dpp.untis_term_id
     FROM deputat_pro_periode dpp
     WHERE dpp.lehrer_id = la.lehrer_id
       AND dpp.gueltig_von <= t.tag
     ORDER BY dpp.gueltig_von DESC
     LIMIT 1)                                AS quell_term_id,
    -- Liegt der Tag innerhalb einer Periode oder in einer fortgeschriebenen Lücke?
    EXISTS (
        SELECT 1 FROM deputat_pro_periode dpp
        WHERE dpp.lehrer_id = la.lehrer_id
          AND t.tag BETWEEN dpp.gueltig_von AND dpp.gueltig_bis
    )                                        AS in_periode
FROM tage t
CROSS JOIN lehrer_aktiv la;

-- Stufe 2: Monatsmittel über die Kalendertage (immer 28/29/30/31 als Nenner)
CREATE OR REPLACE VIEW v_deputat_monat_tagesgenau AS
SELECT
    vdt.lehrer_id,
    hj.id                            AS haushaltsjahr_id,
    EXTRACT(YEAR  FROM vdt.tag)::int   AS jahr,
    EXTRACT(MONTH FROM vdt.tag)::int   AS monat,
    AVG(vdt.deputat_gesamt)          AS deputat_gesamt_tagesgenau,
    COUNT(*)                         AS tage_im_monat,
    -- Repräsentative Periode (Mehrheit der Monatstage)
    mode() WITHIN GROUP (ORDER BY vdt.quell_term_id) AS dominante_term_id
FROM v_deputat_pro_tag vdt
JOIN haushaltsjahre hj
  ON hj.jahr = EXTRACT(YEAR FROM vdt.tag)::int
GROUP BY vdt.lehrer_id, hj.id, EXTRACT(YEAR FROM vdt.tag), EXTRACT(MONTH FROM vdt.tag);
```

Performance-Hinweis: Bei 100 Lehrern × 365 Tagen × 2 SY = ~73 000 Zeilen in der Stufe-1-View, die per LATERAL-LEAD durchsucht werden. Das ist machbar, aber im Hot Path (Stellen-IST-Berechnung) wird daraus eine **Materialized View** mit Refresh am Ende jedes Sync-Laufs.

4.4 Änderungshistorie als View

```
CREATE OR REPLACE VIEW v_deputat_aenderungen AS
SELECT
    a.lehrer_id,
    a.untis_schoolyear_id,
    a.untis_term_id          AS term_alt,
    b.untis_term_id          AS term_neu,
    a.deputat_gesamt         AS gesamt_alt,
    b.deputat_gesamt         AS gesamt_neu,
    b.deputat_gesamt - a.deputat_gesamt AS delta,
    b.gueltig_von            AS wirksam_ab,
    a.gueltig_bis            AS letzter_tag_alt
FROM deputat_pro_periode a
JOIN deputat_pro_periode b
    ON b.lehrer_id            = a.lehrer_id
   AND b.untis_schoolyear_id  = a.untis_schoolyear_id
   AND b.untis_term_id        = a.untis_term_id + 1
WHERE a.deputat_gesamt <> b.deputat_gesamt
   OR a.deputat_ges    <> b.deputat_ges
   OR a.deputat_gym    <> b.deputat_gym
   OR a.deputat_bk     <> b.deputat_bk;
```

Für Bergmann liefert dieser View **automatisch und genau die 4 echten Wechsel** — ohne dass irgendwo "Coverage-Tie-Breaker"-Logik laufen muss.

---

5. Wie sieht das UI für Bergmann dann aus

5.1 Lehrer-Detailseite — Sektion „Periodenverlauf"

```
Bergmann Benjamin (P 600281, GYM)  — Schuljahr 2025/2026

Periode    | Gültig                      | Wochenstd. | Δ
———————————|————————————————————————————|————————————|———————
T1 P1      | 25.08.2025 – 14.09.2025     | 8,0        |
T2 P2      | 15.09.2025 – 21.09.2025     | 8,0        |
T3 P3      | 22.09.2025 – 05.10.2025     | 8,0        |
T4 P4      | 06.10.2025 – 19.10.2025     | 8,0        |
T5 P5      | 20.10.2025 – 02.11.2025     | 8,0        |
T6 P6      | 03.11.2025 – 09.11.2025     | 10,0       | +2,0  ◀ Wechsel
T7 P7      | 10.11.2025 – 16.11.2025     | 10,0       |
…
T11 P11    | 09.02.2026 – 01.03.2026     | 17,0       | +7,0  ◀ Wechsel
…
T14 P13    | 13.04.2026 – 19.04.2026     | 10,0       | -7,0  ◀ Wechsel
…
T17 P15    | 04.05.2026 – 24.05.2026     | 16,0       | +6,0  ◀ Wechsel
T18 P16    | 25.05.2026 – 19.07.2026     | 16,0       |
```

5.2 Sektion „Änderungen" (kompakt)

```
4 Änderungen im Schuljahr 2025/2026

▲ +2,0 WS  ab Mo 03.11.2025  (T5 P5 → T6 P6)         8,0  →  10,0
▲ +7,0 WS  ab Mo 09.02.2026  (T10 P10 → T11 P11)    10,0  →  17,0
▼ -7,0 WS  ab Mo 13.04.2026  (T13 P12b → T14 P13)   17,0  →  10,0
▲ +6,0 WS  ab Mo 04.05.2026  (T16 P14b → T17 P15)   10,0  →  16,0
```

5.3 Sektion „Monatswerte (tagesgenau, für Statistik)"

```
HJ 2026                Tagesgenau    Bisher (DB)    Differenz
──────────────────────────────────────────────────────────────
Jan 2026               10,000        10,0           0
Feb 2026               15,000        17,0           +2,0  ◀
Mär 2026               17,000        17,0           0
Apr 2026               12,800        17,0           +4,2  ◀
Mai 2026               15,419        16,0           +0,58 ◀
Jun 2026               16,000        16,0           0
Jul 2026 (¹)            9,806        16,0           +6,2  ◀
                                                    ─────
                                         Σ Fehler   13,0 WS
```

(¹) Frage: Juli mit 31 Kalendertagen rechnen oder mit 19 Schultagen? **Bitte klären.**

---

6. n8n-Erweiterung

Workflow #223 bekommt **zwei zusätzliche Schritte:**

6.1 Neuer Knoten „Untis-Terms abrufen" (vor dem Lehrer-Knoten)

```
SELECT
    tr.SCHOOLYEAR_ID, tr.TERM_ID, tr.Name AS term_name,
    -- echtes DateFrom
    DATEFROMPARTS(tr.DateFrom/10000, (tr.DateFrom/100)%100, tr.DateFrom%100) AS date_from,
    -- effektives DateTo: LEAD(DateFrom)-1, fallback: echtes DateTo
    DATEADD(DAY, -1, ISNULL(
        DATEFROMPARTS(
            LEAD(tr.DateFrom) OVER (PARTITION BY tr.SCHOOLYEAR_ID ORDER BY tr.TERM_ID)/10000,
            (LEAD(tr.DateFrom) OVER (PARTITION BY tr.SCHOOLYEAR_ID ORDER BY tr.TERM_ID)/100)%100,
            LEAD(tr.DateFrom) OVER (PARTITION BY tr.SCHOOLYEAR_ID ORDER BY tr.TERM_ID)%100
        ),
        DATEFROMPARTS(tr.DateTo/10000, (tr.DateTo/100)%100, tr.DateTo%100)
    )) AS date_to,
    -- b-Perioden-Erkennung: echtes DateTo != Schuljahresende
    CASE WHEN tr.DateTo < (
        SELECT MAX(DateFrom) FROM Terms WHERE SCHOOLYEAR_ID = tr.SCHOOLYEAR_ID AND Deleted = 0
    ) THEN 1 ELSE 0 END AS is_b_period
FROM Terms tr
WHERE tr.Deleted = 0
  AND tr.SCHOOLYEAR_ID IN (
      (SELECT MAX(SCHOOLYEAR_ID) FROM Teacher WHERE Deleted=0),
      (SELECT MAX(SCHOOLYEAR_ID) - 10001 FROM Teacher WHERE Deleted=0)
  )
ORDER BY tr.SCHOOLYEAR_ID, tr.TERM_ID;
```

Diese Daten werden via **neuem Endpoint** `POST /api/untis-terms/sync` an die App geschickt → upsert in `untis_terms`.

6.2 Bestehender Lehrer-Sync wird auf `/api/deputate/sync-v2` umgebogen

Payload-Struktur ändert sich nicht groß — der Endpunkt schreibt zusätzlich in `deputat_pro_periode`:

```
// pro Lehrer pro Term: eine Zeile in deputat_pro_periode upserten
INSERT INTO deputat_pro_periode (
    lehrer_id, untis_schoolyear_id, untis_term_id,
    gueltig_von, gueltig_bis,
    deputat_gesamt, deputat_ges, deputat_gym, deputat_bk,
    stammschule_code, sync_datum
) VALUES (...)
ON CONFLICT (lehrer_id, untis_schoolyear_id, untis_term_id)
DO UPDATE SET ...;
```

Vorteil: Kein Coverage-Tie-Breaker mehr nötig. Jeder Sync schreibt seine Periode rein, Punkt. Werte können nur durch einen weiteren Sync mit demselben `(lehrer, sy, term)` überschrieben werden — eindeutig und idempotent.

---

7. Migrationspfad

Phase 1: Daten parallel mitschreiben (1 Woche)

- [ ] Migration 0008: `untis_terms` + `deputat_pro_periode`
- [ ] n8n-Workflow erweitern (Terms-Sync + Lehrer-Sync schreibt zusätzlich `deputat_pro_periode`)
- [ ] `deputat_monatlich` bleibt unverändert (Doppel-Schreibung) — keine Verhaltensänderung
- [ ] Backfill: aus `Teacher`-Tabelle alle Perioden für SY 2024/25 + 2025/26 importieren

Phase 2: Berechnungen umstellen (1–2 Wochen)

- [ ] Stellen-IST-Berechnung liest aus `v_deputat_monat_tagesgenau` statt `deputat_monatlich`
- [ ] UI-Änderungshistorie liest aus `v_deputat_aenderungen` statt `deputat_aenderungen`
- [ ] Lehrer-Detailseite bekommt Periodenliste-Sektion
- [ ] Stellenplan-Export Anlage 2a verwendet tagesgenaue Monatswerte
- [ ] **Verifikation:** Bergmann + 5 weitere Lehrer mit echten Wechseln händisch nachrechnen

Phase 3: Altsystem zurückbauen (eine Woche, später)

- [ ] `deputat_monatlich` wird zur Materialized View über `v_deputat_monat_tagesgenau`
- [ ] `deputat_aenderungen` wird `_archiv` umbenannt (Historie bleibt erhalten)
- [ ] Coverage-Tie-Breaker-Logik aus Sync-Endpoint entfernen
- [ ] Tests anpassen

---

8. Risiken & offene Fragen

Risiko / Frage	Antwort / Entscheidung
Juli — wie rechnen?	**ENTSCHIEDEN:** Volle Kalendertage (31), gleiche Logik in jedem Monat. Keine Sonderfälle.
Sommerferien-Lücke zwischen T18 (alt SY) und T1 (neu SY)	**ENTSCHIEDEN:** Letzten bekannten Wert fortschreiben — zwischen den Periodenintervallen wird der zuletzt gültige Wert verwendet. Logik in `v_deputat_pro_tag` über `ORDER BY gueltig_von DESC LIMIT 1`.
Termwechsel über Schuljahresgrenze (T18 SY 2025/26 → T1 SY 2026/27)	View `v_deputat_aenderungen` darf nicht auf `term_id + 1` aufbauen, sondern muss per `LEAD() OVER (PARTITION BY lehrer_id ORDER BY gueltig_von)` über alle Periodenwerte hinweg den Folge-Wert finden. (Anpassung der SQL in §4.4.)
Doppelte Personalnummern (Reich Lea / Bergmann B. mit P 600281)	Unabhängig vom Konzept — Match in der App geschieht über `untis_teacher_id`, nicht Personalnummer. Daten sollten in Untis korrigiert werden.
b-Perioden ohne Wertänderung (z. B. P12b/P14b bei Bergmann gleicher Wert)	Werden als eigene Zeile gespeichert (Untis = Quelle der Wahrheit) — im Änderungsview tauchen sie nicht auf, weil Werte gleich bleiben. Korrekt.
Performance Aggregations-View	100 Lehrer × 365 Tage × 2 SY = ~73 000 Zeilen in `v_deputat_pro_tag`. View für Ad-hoc OK; Stellen-IST-Berechnung nutzt **Materialized View** mit Refresh nach Sync.
Migration der bisherigen `deputat_aenderungen`	Bleibt als Archiv-Tabelle (`deputat_aenderungen_archiv`). Der neue View liefert die Wahrheit aus `deputat_pro_periode`, das Archiv dient Forensik.

9. Was uns das bringt — TL;DR

Punkt	Vorher	Nachher
Speicher-Granularität	1 Wert pro Monat	1 Wert pro Periode
Datenverlust bei Coverage-Konflikt	ja (alle außer Sieger)	**nein** (alle Werte erhalten)
Tagesgenaue Monatsberechnung	unmöglich (fehlende Werte)	**trivial** (View)
Änderungserkennung	komplex (Coverage-Tie-Breaker)	**trivial** (Differenz aufeinanderfolgender Terms)
Sync-Reihenfolge spielt eine Rolle	ja (Race Conditions möglich)	**nein** (idempotenter Upsert)
Bergmann Februar/April/Mai/Juli korrekt	nein (bis zu +6,2 WS Fehler)	**ja** (taggenau)
Audit-Trail Wertwechsel	künstliche Zwangs-Erkennung	natürlich (View über zwei aufeinanderfolgende Periodenwerte)

---

10. Entscheidungen (Stand 2026-04-29, mit Dimitri abgestimmt)

#	Frage	Entscheidung
1	Juli rechnen über 31 Kalendertage oder 19 Schultage?	**31 Kalendertage** — gleiche Logik in jedem Monat, keine Sonderfälle.
2	Sommerferien-Lücke fortschreiben oder als 0?	**Fortschreiben** des letzten bekannten Werts.
3	Backfill SY 2024/25 + 2025/26?	**Ja** — sobald das neue Modell live und verifiziert ist.
4	Endpoint-Strategie?	**Neuer Endpoint** (`POST /api/deputate/sync-v2`) — sauber getrennt vom alten.
5	`deputat_monatlich` Zukunft?	Während Phase 1+2 parallel mitschreiben; in Phase 3 als **Materialized View** über `v_deputat_monat_tagesgenau` ersetzen, damit alle bestehenden Berechnungen unverändert weiter funktionieren.
6	Korrektur-Layer für tatsächliches Datum?	**Eigene Tabelle** `deputat_aenderung_korrekturen` (Migration 0010) — eine Korrektur pro Term-Wechsel-Ziel, deckt sowohl gehaltsrelevante als auch Verteilungs-Änderungen ab.

Phase 1 + Korrektur-Layer sind implementiert (2026-04-29).

---

11. Korrektur-Layer für Sachbearbeiter (Migration 0010)

Problem

Untis erlaubt aus technischen Gründen Periodenwechsel **nur zum Montag**. Im Personalbestand passiert die echte Änderung aber an jedem Wochentag — eine Reduktion ab **Mi 04.02.2026** wird in Untis als **Mo 09.02.2026** abgebildet, mit 3 Tagen Versatz.

Für eine korrekte Refinanzierung nach **§ 3 Abs. 1 FESchVO** und die Gehaltsabrechnung muss der **echte Stichtag** tagesgenau erfasst werden — und das ist die Aufgabe des Sachbearbeiters.

Lösung: Korrektur-Layer als eigene Tabelle

Untis (`deputat_pro_periode`) bleibt 1:1 die Quelle der Wahrheit. Die menschliche Korrektur lebt in einer **eigenen Tabelle**, die beim Untis-Sync nie angetastet wird:

```
CREATE TABLE deputat_aenderung_korrekturen (
    id                        SERIAL PRIMARY KEY,
    lehrer_id                 INTEGER NOT NULL REFERENCES lehrer(id) ON DELETE CASCADE,
    sy_alt                    INTEGER NOT NULL,
    term_id_alt               INTEGER NOT NULL,
    sy_neu                    INTEGER NOT NULL,
    term_id_neu               INTEGER NOT NULL,
    tatsaechliches_datum      DATE NOT NULL,           -- echter Stichtag
    korrigiert_von            VARCHAR(100),
    korrigiert_am             TIMESTAMPTZ DEFAULT now(),
    bemerkung                 TEXT,
    UNIQUE (lehrer_id, sy_neu, term_id_neu),         -- max 1 Korrektur pro Wechsel-Ziel
    FOREIGN KEY (sy_alt, term_id_alt)
        REFERENCES untis_terms(school_year_id, term_id),
    FOREIGN KEY (sy_neu, term_id_neu)
        REFERENCES untis_terms(school_year_id, term_id)
);
```

Schlüssel-Idee: Pro Term-zu-Term-Wechsel max. eine Korrektur. Sie ist immer an die **neue** Periode geheftet (das ist die, deren Start verschoben wird).

Wirkung auf die Views

`v_deputat_pro_tag` joined die Korrekturen und nutzt das **effektive Wirksamkeitsdatum** (`COALESCE(tatsaechliches_datum, gueltig_von)`) für die Tag-Zuordnung — die alte Periode reicht entsprechend kürzer/länger, die neue startet entsprechend früher/später.

`v_deputat_aenderungen` joined die Korrekturen ebenfalls und liefert pro Wechsel:

- `wirksam_ab` (Untis-Montag — bleibt für Audit-Zwecke sichtbar)
- `tatsaechliches_datum` (NULL falls unkorrigiert)
- `effektiv_wirksam_ab` (was die Berechnung wirklich verwendet)
- `hat_korrektur` , `korrigiert_von` , `korrigiert_am` , `bemerkung`

→ `v_deputat_monat_tagesgenau` rechnet automatisch mit den Korrekturen, weil sie auf der korrigierten Tages-View aufbaut.

Beispiel: Bergmann mit Korrektur

Angenommen, T10 → T11 (Reduktion 10 → 17 WS) wird **Mi 04.02.2026** gesetzt statt Untis- `Mo 09.02.2026`:

Monat	Vor Korrektur	Nach Korrektur
Feb 2026	(8 × 10 + 20 × 17) / 28 = **15,000**	(3 × 10 + 25 × 17) / 28 = **16,250**

→ 1,25 Wochenstunden mehr im Februar, weil 5 Tage früher reduziert.

Eine Tabelle für beide Fälle

Die Korrektur-Tabelle deckt **gehaltsrelevante Wechsel** (Gesamt-WS ändert sich) UND **reine Schul-Verteilungswechsel** (GES↔GYM↔BK) ab. Der `tatsaechliches_datum`-Wert ist in beiden Fällen die gleiche Information (echter Stichtag), und die View-Logik unterscheidet nicht — beide profitieren gleichermaßen von der tagesgenauen Berechnung.

UI-Erweiterung in PeriodenModellCard

In der Tabelle „Echte Wertwechsel" eine zusätzliche Spalte **„Tats. Datum"**:

```
Untis-Montag   | Tats. Datum   | Term-Wechsel | Wert alt → neu | Δ
Mo 09.02.2026  | Mi 04.02.2026 ✎🗑 | T10 → T11    | 10,0 → 17,0    | +7,0
Mo 13.04.2026  | Datum setzen ✎    | T13 → T14    | 17,0 → 10,0    | -7,0
```

- **„Datum setzen"** öffnet ein Inline-Date-Picker, schreibt via Server-Action `korrigierePeriodeWirksamkeitAction`
- ✎ öffnet vorhandene Korrektur zum Editieren
- 🗑 entfernt die Korrektur (Wechsel gilt dann wieder zum Untis-Montag)

Migration aus altem Modell

Bestehende `deputat_aenderungen.tatsaechliches_datum`-Einträge werden im Migration-Schritt 0010 nach `deputat_aenderung_korrekturen` übertragen. Mapping:

- `(lehrer_id, term_id_alt, term_id_neu)` aus alter Tabelle
- `school_year_id` aus dem zugehörigen `deputat_pro_periode`-Eintrag erschließen (gleicher Lehrer + gleiche `term_id_neu` + Periode-Jahr passt zum HJ-Jahr)
- `korrigiert_von`, `korrigiert_am`, `bemerkung = 'Backfill aus deputat_aenderungen #<id>'`

Auf Konflikt (extrem unwahrscheinlich) gewinnt der jüngste Eintrag (`ORDER BY geaendert_am DESC`).

Audit & Auditierbarkeit

Jeder Save/Delete schreibt einen Eintrag in `audit_log`:

- `tabelle = 'deputat_aenderung_korrekturen'`

- `aktion = 'INSERT' | 'UPDATE' | 'DELETE'`
- `alte_werte` / `neue_werte` mit `tatsaechliches_datum` und `bemerkung`
- `geaendert_von` = aktueller User aus `requireWriteAccess()`

Damit ist nachvollziehbar, wer wann welches Datum gesetzt/geändert hat — wichtig für Compliance-Audits.